STAT:5400 Section 5.1 — AI Agents

Boxiang Wang

2026-11-04

0.1 What an agent is

An **agent** is a program in which a language model decides the next step.

- The model gets a task and a list of functions it may call. The functions are the **tools**.
- The model asks for a tool. Our code runs it and sends the result back.
- Repeat until the model gives the answer. This is the **loop**.

0.1.1 The basic elements

1. **Model.** The API call of Section 2.4.
2. **Tools.** Ordinary functions. Each one has a one-sentence description for the model.
3. **Instructions.** The system prompt: what to do, in which order.
4. **Loop.** Ask, run the tool, show the result, repeat, stop.

The **receipt** is the list of all tool calls, with arguments and results. We keep it to check the answer.

0.1.2 One call, a workflow, or an agent

A **model call** is one request to the language model: we send text, it sends text back. `chat$chat("...")` in Section 2.4 is one model call. There are three ways to use a model for a task. Take the task of this page: build the R package of Tech Guide 3.

- **One model call.** "Write an R package named pvals with these three functions." The model writes code as text. Nothing is saved, built, or checked.
- **A workflow.** A fixed recipe written by us. Step 1: ask the model for the code. Step 2: our script saves the files and runs `R CMD check`. Step 3: ask the model to fix the errors. The model never chooses the next step. We do.

- **An agent.** We give the model the tools (write a file, run R code) and the task. The model chooses the steps: write `DESCRIPTION`, write the R file, run `roxygenise`, build, check, fix, check again. We did not decide the order. It did.

Use the simplest one that works. Build an agent only when the steps are not known in advance.

0.2 Build an agent

On this page we write the specification of an agent, and a coding agent writes the script from it: Claude Code on the instructor's laptop, Copilot in the Codespace. The one-tool agent below is the only script we read by hand.

0.2.1 The smallest agent

Run in Codespace: a real model needs a key. Set `OPENAI_API_KEY` once in `~/.Renviron`, or as a Codespaces secret.

One tool and one task. `ellmer` runs the loop inside `chat$chat()`.

```
library(ellmer)

# 1. a tool: an ordinary R function
run_r <- function(code) paste(capture.output(print(eval(parse(text = code)))), collapse = "\n

# 2. its description: what the model reads instead of the code
tool_run <- tool(run_r, "Run one piece of R code and return what it prints.",
                 arguments = list(code = type_string("The R code to run.")))

# 3. the model, with instructions, and the tool attached
chat <- chat_openai(system_prompt = "Use the tool to compute. Never guess a number.")
chat$register_tools(list(tool_run))

# 4. print the receipt as it happens
chat$on_tool_request(function(request) cat("-> run_r:", request@arguments$code, "\n"))
chat$on_tool_result(function(result)   cat("<-", result@value, "\n"))

# 5. one task. ellmer runs the loop: ask, act, observe, repeat, until the model answers.
chat$chat("What is the 0.975 quantile of the t distribution with 10 degrees of freedom?")
```

The five comments are the four elements of Section 1.1. Comments 1 and 2 are the tool, comment 3 is the model and the instructions, and comments 4 and 5 are the loop, which `ellmer` runs for us, and the receipt it prints.

0.2.2 The specification

A **specification** is a short text that says what the agent must do. It has four parts: the task, the tools, the instructions, and the repo layout. Here it is for an agent that writes an R package, the package of Tech Guide 3:

```
Task
  An R package named pvals that computes two-sided p-values from a test statistic:
  pval_z(z) for a standard normal statistic, pval_t(t, df) for a t statistic,
  and pval_chisq(x, df) for a chi-square statistic (upper tail).

Tools
  write_file(path, content): write a text file inside the sandbox; create folders as needed.
  read_file(path): read a text file inside the sandbox and return its content.
  list_files(): list every file in the sandbox, one per line.
  run_r(code): run R code in a fresh R session inside the sandbox and return what it printed
    High risk: it runs any R code.

Instructions (the system prompt)
  You are an experienced R package developer. Build the package inside the sandbox,
  using only the tools, in this order.
  1. Write pvals/DESCRIPTION and pvals/R/pvals.R with roxygen comments.
  2. Run roxygen2::roxygenise('pvals') to create NAMESPACE and man/.
  3. Run R CMD build, then R CMD check --no-manual on the .tar.gz file.
  4. Read the check output. Fix every ERROR and WARNING, then check again,
     until the check ends with Status: OK.
  5. Try each function once with run_r. Do not install the package.
  6. Report the files you wrote, the final check status, and the path of the .tar.gz file.
  Never guess what a command printed; read the tool result.
  Do not write files outside the package folder.

Repo layout
  README.md    what the agent does, which key it needs, how to run it
  agent.R      the tools, their descriptions, the instructions, and chat$chat()
  sandbox/     the folder the agent works in; the package it builds goes here
  receipts/    one saved .rds file per run
```

Copy the block and change the four parts for your own task. Note that each tool is one line:

its name, its arguments, and one sentence. If you cannot write that line, the model cannot use the tool.

0.2.3 The prompt for the coding agent

Paste the specification, then this prompt, into Claude Code or Copilot:

```
Build this repository from the specification above, in R with the ellmer package.
- The tools are plain R functions. Each one gets a tool() description that says
  what it does and what each argument means.
- write_file and read_file must refuse any path outside sandbox/.
- Print every tool request and every tool result as it happens
  (chat$on_tool_request and chat$on_tool_result).
- After the run, save chat$get_turns() and chat$get_tokens() to receipts/ with saveRDS.
- Read OPENAI_API_KEY from the environment. Never write a key into any file.
- Show me agent.R before you run it.
```

Read the script it shows you, then run it and keep the printed receipt. Our own version is `run/s5p1_pkg_agent.R`; on Nov 4, with `gpt-5.4`, it took five tool calls and the check ended with `Status: OK`.

0.2.4 The receipt checklist

The receipt is the printed list of tool requests and results. Read it before you accept the package:

- Is there an `R CMD check` call in the receipt? If the model reports `Status: OK` without one, the report is a guess. If there is one, does the report match its output?
- When a check failed, what did it change before the next check?
- Did it write a file outside the package folder `pvals/`? The instructions forbid this, but `write_file` only refuses paths outside the sandbox, so such a file passes the tool.
- Did every number and name in its report come from a tool result you can see? In our run the model invented an author name and a `RoxygenNote` version. Both came from memory, not from a tool.

A loop needs an exit, and a person needs a way to step in:

- There are four exits: a final answer, a maximum number of steps, an error the loop cannot recover from, and a structured final output, where the model returns an object of a declared type (structured output).

- `run_r` is the high-risk tool: it runs any R code, anywhere on the machine. Set `ASK <- TRUE` at the top of the script to approve each `run_r` call yourself. This is **human intervention**.
- Hand control back to a person after several failures, or before a high-risk action such as deleting a file.

0.2.5 Frameworks

In practice you do not write the loop. A **framework**, a package that runs the loop for you, does it.

- **`ellmer`** (R): `tool()` describes, `chat$register_tools()` attaches, `chat$chat()` loops. The receipt is `chat$get_turns()`.
- **OpenAI Agents SDK** and **Claude Agent SDK** (Python): the same loop as a library. The Claude Agent SDK is the loop of Claude Code, with built-in tools to read and edit files, run commands, and search the web.
- **MCP** (Model Context Protocol): a standard format for describing a tool. Write the tool once in this format, and any model that supports MCP can call it.

In a new framework, find the four elements and the exit conditions.

0.3 Supervise an agent

0.3.1 A stand-in agent, for simulation

To study an agent statistically we need one that runs thousands of times for free. So here the model is a plain R function that follows a script, and the tools are two bootstrap functions for the task of HW 9: the mean failure time of the air-conditioning data and its bootstrap standard error.

Run this block first. Everything below uses it.

0.3.1.1 R

```
aircondit <- c(3, 5, 7, 18, 43, 85, 91, 98, 100, 130, 230, 487)   # hours between failures, h

#' Describe a data vector: its length, mean, and median.
#' Call this before bootstrapping.
#' @param name Name of the vector, for example "aircondit".
describe <- function(name) {
  x <- get(name)
```

```
  sprintf("%d values; mean %.1f; median %.1f", length(x), mean(x), median(x))
}

# set the seed for one computation only, then put the random number state back
with_seed <- function(seed, expr) {
  old <- if (exists(".Random.seed", globalenv())) get(".Random.seed", globalenv())
  set.seed(seed)
  on.exit(if (!is.null(old)) assign(".Random.seed", old, globalenv()))
  expr
}

#' Bootstrap standard error of a statistic of a data vector.
#' @param name Name of the vector.
#' @param stat The statistic: "mean" or "median".
#' @param B Number of bootstrap samples.
#' @param seed Seed for the random numbers, so the result can be repeated.
boot_se <- function(name, stat, B = 2000, seed = 5400) {
  x <- get(name)
  f <- match.fun(stat)
  est <- with_seed(seed, replicate(B, f(sample(x, replace = TRUE))))
  c(estimate = f(x), se = sd(est))
}

tools <- list(describe = describe, boot_se = boot_se)
```

```
describe("aircondit")
boot_se("aircondit", stat = "mean", B = 2000)
```

0.3.1.2 Python

```
import numpy as np

aircondit = np.array([3, 5, 7, 18, 43, 85, 91, 98, 100, 130, 230, 487])   # hours between fa
DATA = {"aircondit": aircondit}

def describe(name):
    """Describe a data vector: its length, mean, and median.
    Call this before bootstrapping."""
    x = DATA[name]
    return f"{len(x)} values; mean {x.mean():.1f}; median {np.median(x):.1f}"
```

```
def boot_se(name, stat, B=2000, seed=5400):
    """Bootstrap standard error of a statistic ("mean" or "median") of a data vector."""
    x = DATA[name]
    f = {"mean": np.mean, "median": np.median}[stat]
    rng = np.random.default_rng(seed)
    est = [f(rng.choice(x, size=len(x), replace=True)) for _ in range(B)]
    return {"estimate": round(float(f(x)), 4), "se": round(float(np.std(est, ddof=1)), 4)}

tools = {"describe": describe, "boot_se": boot_se}
```

```
print(describe("aircondit"))
print(boot_se("aircondit", stat="mean", B=2000))
```

The tools are plain functions. The `#'` lines are the description, as in Tech Guide 3. The second block calls the tools by hand, with no model.

Note that `boot_se` does not call `set.seed` directly. `with_seed` sets the seed for the bootstrap only and then restores the random number state. Otherwise every tool call would reset the random numbers of the whole session.

Here is the loop that `chat$chat()` runs for us in Section 2, written out in full. The `model` argument is a function: it takes the history so far and returns a tool request or a final answer.

0.3.1.3 R

```
show_result <- function(r) {
  if (is.character(r)) return(paste(r, collapse = " "))
  paste(names(r), signif(r, 4), sep = " = ", collapse = ", ")
}

run_agent <- function(model, tools, task, max_steps = 6) {
  history <- task
  receipt <- list()
  for (step in seq_len(max_steps)) {
    action <- model(history, names(tools))           # 1. ask the model what to do
    if (!is.null(action$final)) {                    # 2. it answered: stop
      receipt[[step]] <- data.frame(step, tool = "(answer)", args = "",
                                    result = action$final)
      return(list(answer = action$final, receipt = do.call(rbind, receipt), steps = step))
    }
    result <- tryCatch(                              # 3. run the tool it asked for
      do.call(tools[[action$tool]], action$args),
```

```
      error = function(e) paste("ERROR:", conditionMessage(e)))
    shown <- show_result(result)
    receipt[[step]] <- data.frame(step, tool = action$tool,
                                  args = deparse1(action$args), result = shown)
    history <- c(history, paste0(action$tool, ": ", shown))  # 4. show the result
  }
  # 5. out of steps
  list(answer = NA, receipt = do.call(rbind, receipt), steps = max_steps)
}
```

0.3.1.4 Python

```
def run_agent(model, tools, task, max_steps=6):
    history, receipt = [task], []
    for step in range(1, max_steps + 1):
        action = model(history, list(tools))        # 1. ask the model what to do
        if "final" in action:                       # 2. it answered: stop
            receipt.append((step, "(answer)", "", action["final"]))
            return {"answer": action["final"], "receipt": receipt, "steps": step}
        try:                                        # 3. run the tool it asked for
            result = tools[action["tool"]](**action["args"])
        except Exception as e:
            result = f"ERROR: {e}"
        receipt.append((step, action["tool"], str(action["args"]), str(result)))
        history.append(f"{action['tool']}: {result}")  # 4. show the result
    return {"answer": None, "receipt": receipt, "steps": max_steps}   # 5. out of steps

def show(receipt):
    for row in receipt:
        print(*row, sep=" | ")
```

- The tool call (comment `3.` in the code) is inside `tryCatch`. If the tool fails, the error message goes back to the model as text, and the model can try again.
- `max_steps` is the safety exit. Without it, a confused model loops forever, and each turn costs money.

The model is a plain function with a fixed plan, so the loop runs without a key.

0.3.1.5 R

```
scripted <- function(history, tool_names) {
  done <- length(history) - 1                 # how many tool results so far
  if (done == 0) return(list(tool = "describe", args = list(name = "aircondit")))
  if (done == 1) return(list(tool = "boot_se",
    args = list(name = "aircondit", stat = "mean", B = 2000)))
  list(final = paste("Bootstrap SE of the mean:",
    sub(".*se = ([^,]+).*", "\\1", tail(history, 1))))
}
```

```
out <- run_agent(scripted, tools, "Estimate the mean failure time and its bootstrap standard
out$answer
```

0.3.1.6 Python

```
def scripted(history, tool_names):
    done = len(history) - 1                     # how many tool results so far
    if done == 0:
        return {"tool": "describe", "args": {"name": "aircondit"}}
    if done == 1:
        return {"tool": "boot_se", "args": {"name": "aircondit", "stat": "mean", "B": 2000}}
    se = history[-1].split("'se': ")[1].rstrip("}")
    return {"final": "Bootstrap SE of the mean: " + se}
```

```
out = run_agent(scripted, tools, "Estimate the mean failure time and its bootstrap standard e
print(out["answer"])
```

💡 Predict: how many rows will the receipt have, and what is in the last one?

0.3.1.7 R

```
out$receipt
```

0.3.1.8 Python

```
show(out["receipt"])
```

The receipt is the agent's transcript. The rest of this section reads it.

0.3.2 An agent's success rate

- Run a real agent on the same task twice and you can get two different receipts. The model draws its output at random (sampling controls), so which tool it calls, and with what arguments, is random too.
- So whether it succeeds is random. We estimate its **probability of success**, give it a confidence interval (Section 3.2), and choose the number of runs (Section 3.3).

Here is a stand-in model that makes the same kinds of mistakes a real model makes. Sometimes it answers without bootstrapping. Sometimes it bootstraps the median when the task asked for the mean. Sometimes it misspells the statistic, gets an error, and recovers.

0.3.2.1 R

```
sloppy <- function(history, tool_names) {
  done <- length(history) - 1
  u <- runif(1)
  if (done == 0) return(list(tool = "describe", args = list(name = "aircondit")))
  if (grepl("ERROR", tail(history, 1)))          # recover from an error
    return(list(tool = "boot_se", args = list(name = "aircondit", stat = "mean", B = 2000)))
  if (done == 1) {
    # answers without bootstrapping
    if (u < 0.10) return(list(final = "Bootstrap SE of the mean: about 40."))
    # the wrong statistic
    if (u < 0.25) return(list(tool = "boot_se",
                              args = list(name = "aircondit", stat = "median", B = 2000)))
    # a typo in the statistic
    if (u < 0.32) return(list(tool = "boot_se",
                              args = list(name = "aircondit", stat = "maen", B = 2000)))
    return(list(tool = "boot_se", args = list(name = "aircondit", stat = "mean", B = 2000)))
  }
  list(final = paste("Bootstrap SE of the mean:",
    sub(".*se = ([^,]+).*", "\\1", tail(history, 1))))
}
```

```
set.seed(5400)   # run this block once
```

```
run_agent(sloppy, tools, "Estimate the mean failure time and its bootstrap SE.")$receipt
```

0.3.2.2 Python

```
rng = np.random.default_rng(5400)

def sloppy(history, tool_names):
    done = len(history) - 1
    u = rng.uniform()
    if done == 0:
        return {"tool": "describe", "args": {"name": "aircondit"}}
    if history[-1].startswith("boot_se: ERROR"):        # recover from an error
        return {"tool": "boot_se", "args": {"name": "aircondit", "stat": "mean", "B": 2000}}
    if done == 1:
        if u < 0.10:                                     # answers without bootstrapping
            return {"final": "Bootstrap SE of the mean: about 40."}
        if u < 0.25:                                     # the wrong statistic
            return {"tool": "boot_se", "args": {"name": "aircondit", "stat": "median", "B": 
        if u < 0.32:                                     # a typo in the statistic
            return {"tool": "boot_se", "args": {"name": "aircondit", "stat": "maen", "B": 20
        return {"tool": "boot_se", "args": {"name": "aircondit", "stat": "mean", "B": 2000}}
    se = history[-1].split("'se': ")[1].rstrip("}")
    return {"final": "Bootstrap SE of the mean: " + se}
```

```
show(run_agent(sloppy, tools, "Estimate the mean failure time and its bootstrap SE.")["recei
```

Run the seed block once, then rerun only the last block several times. (In Python, the seed is set in the block that defines `sloppy`; rerun only the last block.) Sometimes the receipt is correct and sometimes it is not. To estimate how often, we repeat the run many times, as in the simulation studies of Section 3.2.

0.3.3 Success rate, with an interval

0.3.3.1 R

```
truth <- unname(boot_se("aircondit", "mean", B = 2000, seed = 5400)["se"])
is_right <- function(answer) {
  s <- suppressWarnings(as.numeric(sub(".*: ", "", answer)))
  isTRUE(abs(s - truth) < 0.05)
}

set.seed(5400)
runs <- replicate(200, {
```

```
  r <- run_agent(sloppy, tools, "Estimate the mean failure time and its bootstrap SE.")
  c(ok = is_right(r$answer), steps = r$steps)
})
ok <- runs["ok", ]
mean(ok)
```

```
# Wilson interval for the success rate, Section 3.2
prop.test(sum(ok), length(ok), correct = FALSE)$conf.int
```

```
table(steps = runs["steps", ])
```

0.3.3.2 Python

```
truth = boot_se("aircondit", "mean", B=2000, seed=5400)["se"]

def is_right(answer):
    try:
        return abs(float(answer.split(": ")[1]) - truth) < 0.05
    except Exception:
        return False

task = "Estimate the mean failure time and its bootstrap SE."
rng = np.random.default_rng(5400)
runs = [run_agent(sloppy, tools, task) for _ in range(200)]
ok = np.array([is_right(r["answer"]) for r in runs])
print(ok.mean())
```

```
# Wilson interval, by the formula from Section 3.2
n, p, z = len(ok), ok.mean(), 1.959964
center = (p + z**2 / (2 * n)) / (1 + z**2 / n)
half = z * np.sqrt(p * (1 - p) / n + z**2 / (4 * n**2)) / (1 + z**2 / n)
print(center - half, center + half)
```

```
steps, counts = np.unique([r["steps"] for r in runs], return_counts=True)
print(dict(zip(steps.tolist(), counts.tolist())))
```

- The agent is right about three quarters of the time, and the interval is about plus or minus six points.
- Report the success rate with its confidence interval, not a single successful run.

💡 Predict: the table of steps has three values. Which kind of sloppy run produces each one?

Answer

Reading the steps table: only the wrong-statistic run looks clean and is wrong. Two steps: the agent answered after `describe` without bootstrapping. Three steps: the normal run, or the median run, which also takes three steps and is *wrong* while looking clean. Four steps: the typo, an error, and a recovery.
The median case is the dangerous one. Its receipt is the same length as a correct run and every tool call succeeded. Only reading the arguments catches it. That is the audit task below.

0.3.4 Audit the agent

Three receipts from the bootstrap agent. The task was the same each time: the mean failure time and its bootstrap standard error. Each receipt has a problem. None of them threw an error.

✋ By hand: read each receipt and write one sentence saying what went wrong and which course principle it violates. Done when you can name all three without running anything. No AI for this block.

0.3.4.1 R

```
receipt_1 <- data.frame(
  step = 1:3,
  tool = c("describe", "boot_se", "(answer)"),
  args = c('list(name = "aircondit")',
           'list(name = "aircondit", stat = "median", B = 2000)', ""),
  result = c("12 values; mean 108.1; median 88.0", "estimate = 88, se = 28.78",
             "Bootstrap SE of the mean: 28.78"))
receipt_1
```

```
receipt_2 <- data.frame(
  step = 1:2,
  tool = c("describe", "(answer)"),
  args = c('list(name = "aircondit")', ""),
  result = c("12 values; mean 108.1; median 88.0",
             "Bootstrap SE of the mean: 39.3 (the standard deviation divided by the square r
receipt_2
```

```
receipt_3 <- data.frame(
  step = 1:4,
  tool = c("describe", "boot_se", "boot_se", "(answer)"),
  args = c('list(name = "aircondit")',
           'list(name = "aircondit", stat = "mean", B = 2000, seed = 5400)',
           'list(name = "aircondit", stat = "mean", B = 2000, seed = 5400)', ""),
  result = c("12 values; mean 108.1; median 88.0", "estimate = 108.1, se = 36.77",
             "estimate = 108.1, se = 36.77",
             "Bootstrap SE of the mean: 36.77, confirmed by two independent bootstrap runs.")
receipt_3
```

0.3.4.2 Python

```
receipt_1 = [
    (1, "describe", "{'name': 'aircondit'}", "12 values; mean 108.1; median 88.0"),
    (2, "boot_se",  "{'name': 'aircondit', 'stat': 'median', 'B': 2000}", "{'estimate': 88.0
    (3, "(answer)", "", "Bootstrap SE of the mean: 28.78")]
show(receipt_1)
```

```
receipt_2 = [
    (1, "describe", "{'name': 'aircondit'}", "12 values; mean 108.1; median 88.0"),
    (2, "(answer)", "", "Bootstrap SE of the mean: 39.3 (the standard deviation divided by t
show(receipt_2)
```

```
receipt_3 = [
    (1, "describe", "{'name': 'aircondit'}", "12 values; mean 108.1; median 88.0"),
    (2, "boot_se",  "{'name': 'aircondit', 'stat': 'mean', 'B': 2000, 'seed': 5400}", "{'esti
    (3, "boot_se",  "{'name': 'aircondit', 'stat': 'mean', 'B': 2000, 'seed': 5400}", "{'esti
    (4, "(answer)", "", "Bootstrap SE of the mean: 36.77, confirmed by two independent boots
show(receipt_3)
```

i Solution

Receipt 1: the wrong statistic. The task asked for the mean; the tool was called with `stat = "median"`. The number 28.78 is the standard error of the median. Every step succeeded, and the answer is wrong. Principle: read the arguments, not just the results. **Receipt 2: the number was never bootstrapped.** There is no `boot_se` call. The model computed $s/\sqrt{n}$ from memory, the standard error formula of Section 3.2, and called it a bootstrap standard error. It is close to the bootstrap value, which makes the error hard to see. Principle: every number must come from a tool call. A number with no tool

call behind it is a guess.
Receipt 3: false confirmation. The two `boot_se` calls use the same seed, so they are the same computation twice. Two identical numbers are not two independent bootstrap runs. Principle: replication means a *different* sample or method; here, a different seed. The answer is correct, but the claim of confirmation is not.
Two of the three receipts give a reasonable number. An audit checks how the answer was computed, not only the answer.

0.3.5 References

- Anthropic (December 2024). *Building effective agents*.
- OpenAI (2025). *A practical guide to building agents*.
- Wickham, H. and others. *Tool calling* in `ellmer`.
- Model Context Protocol.